AutoCare Insight: Vehicle Service Management & Predictive Maintenance System

DBMS Experiential Learning — Level 3 Comprehensive Project Report

Department of Computer Science and Engineering
PES University, Bengaluru, India
Course: Database Management Systems (DBMS Level 3)
Academic Year: 2025–2026
Document Classification: Final Project Technical Submission

Executive Summary / Abstract

AutoCare Insight is an enterprise-grade vehicle service lifecycle management, parts inventory control, and predictive maintenance platform engineered to resolve operational fragmentation in modern automotive workshops. Addressing the distinct architectural needs of automotive service operations, the platform implements a **Dual-Database Architecture**:

1. **Relational Paradigm (MySQL 8.0)**: Guarantees strict ACID transaction properties, referential integrity, and ledger accountability across 13 interrelated relational entities handling appointment scheduling, service order dispatching, mechanic assignments, spare parts inventory deduction, and automated 18% GST billing.
2. **Document Paradigm (MongoDB 8.2)**: Manages semi-structured, polymorphic vehicle inspection reports with dynamic multi-point diagnostic findings, OBD-II diagnostic trouble codes (DTCs), and technician recommendations, validated at database level via JSON Schema.
3. **Semantic Vector Search Engine**: Implements an in-memory 64-dimensional dense semantic embedding space normalized to unit length, enabling natural-language diagnostic similarity searches using Cosine Similarity over both curated diagnostic symptom benchmarks and historical MongoDB inspection records.

The system is deployed on a modern **Next.js 14 App Router** frontend and TypeScript backend, featuring Role-Based Access Control (RBAC) across three distinct enterprise personas (`Admin`, `Service Advisor`, `Mechanic`), and includes an interactive educational SQL query runner executing 20 distinct relational queries, a 7-pipeline MongoDB analytical aggregation engine, and automated stock deduction triggers.

Table of Contents

1. **Section 1: Project Overview and Abstract**
2. **Section 2: Problem Statement, Motivation, and Objectives**

3. **Section 3: Requirements Specification and Scope**
4. **Section 4: System Architecture and Technology Stack**
5. **Section 5: Database Design and ER Diagram**
6. **Section 6: Relational Database Schema and Data Dictionary**
7. **Section 7: MongoDB Schema and Document Design**
8. **Section 8: Data Import, Validation, and Integrity**
9. **Section 9: Backend API and Authentication/RBAC**
10. **Section 10: Frontend Design and Feature Workflows**
11. **Section 11: Advanced SQL, MongoDB Aggregations, and Vector Search**
12. **Section 12: Testing, Verification, Results, and Limitations**
13. **Section 13: Installation, User Guide, Conclusion, and Future Enhancements**

Section 1: Project Overview and Abstract

Modern automotive service centers grapple with severe operational silos. Relational transactional workloads—such as inventory stock control, service order accounting, labor billing, and appointment booking—mandate strict atomicity, zero tolerance for financial discrepancies, and high referential integrity. Conversely, vehicle health assessments, multi-point diagnostic inspections, and sensor telemetry exhibit highly variable, semi-structured properties where rigid tabular schemas lead to excessive null values and schema bloat.

AutoCare Insight harmonizes these paradigms through an architectural separation of concerns:

- **Transactional Core (MySQL 8.0)**: Manages 13 relational entities with 18 Foreign Key constraints, stored procedures for scheduling and billing, and event-driven triggers for real-time inventory deductions.
- **Diagnostic Document Store (MongoDB 8.2)**: Accommodates polymorphic multi-point inspection checkpoints across varied automotive subsystems (Engine, Transmission, Brakes, Suspension, Battery, Electrical) with JSON Schema validation and aggregation analytics.
- **Diagnostic Vector Search**: A 64-dimensional dense vector search engine that maps natural language symptom queries to historical repair documents using Cosine Similarity.
- **Full-Stack Application Layer**: A Next.js 14 application built with TypeScript, Tailwind CSS, secure HTTP-only JWT sessions, and RBAC authorization.

Section 2: Problem Statement, Motivation, and Objectives

2.1 Problem Statement

Conventional workshop management applications suffer from three critical bottlenecks:

1. **Relational Over-Engineering for Unstructured Diagnostics**: Forcing vehicle inspection checklists into rigid tabular schemas results in sparse tables, high schema maintenance overhead, and difficulty accommodating emerging EV/hybrid powertrain components.
2. **Disconnected Inventory and Labor Accounting**: Delays between technician part consumption in service bays and front-desk invoice generation lead to inventory drift, unbilled parts, and stockouts.

3. **Keyword-Dependent Symptom Search**: Workshop advisors and mechanics cannot effectively query historical repair archives using natural language complaints (e.g., "*car shudders under acceleration with clicking noise*"), leading to repetitive diagnostic cycles and redundant labor.

2.2 Project Objectives

- **ACID Transaction Guarantees**: Enforce strict atomicity and financial consistency for vehicle bookings, service execution, and billing settlements.
- **Automated Inventory Control**: Implement database-level triggers that decrement physical stock upon part allocation and automatically flag items falling below safety reorder thresholds.
- **Polymorphic Diagnostic Storage**: Provide dynamic multi-point inspection capture with schema validation, DTC logging, and health score calculations.
- **Semantic Diagnostic Retrieval**: Provide a dense vector similarity search engine linking symptom queries to historical inspection records.
- **Enterprise RBAC**: Ensure strict role separation between administrative staff, service advisors, and shop-floor mechanics.

Section 3: Requirements Specification and Scope

3.1 Functional Requirements (FR)

- **FR-01 (Customer & Vehicle Fleet Directory)**: Maintain customer profiles, loyalty tiers, and vehicle registrations linked to technical catalogue specifications.
- **FR-02 (Alternative Fuel & EV Catalogue)**: Ingest and browse 3,229 Department of Energy vehicle models with side-by-side fuel economy and range comparison.
- **FR-03 (Service Booking with Conflict Detection)**: Schedule appointments via stored procedures verifying that vehicles have no conflicting active appointments.
- **FR-04 (Workshop Bay Job Dispatching)**: Manage technician assignment, job status transitions (`assigned` → `in_progress` → `completed`), and odometer verification.
- **FR-05 (Trigger-Driven Part Allocation)**: Charge parts to active service jobs with automated inventory stock decrementing and audit logging.
- **FR-06 (Polymorphic MongoDB Inspection Reports)**: Author, validate, and retrieve multi-point diagnostic inspection reports linked to MySQL job IDs.
- **FR-07 (Automated Tax Invoicing & Settlement)**: Generate invoices with 18% GST and itemized labor/parts, updating settlement status via triggers upon payment recording.
- **FR-08 (MongoDB Analytical Aggregations)**: Compute fleet health trends, subsystem failure frequencies, and cost exposures using multi-stage aggregation pipelines.
- **FR-09 (Semantic Vector Search)**: Search diagnostic symptoms and findings using 64-dimensional dense semantic vectors and Cosine Similarity.
- **FR-10 (Interactive SQL Query Runner)**: Provide a read-only query catalog executing 20 educational SQL queries demonstrating joins, subqueries, CTEs, and window functions.

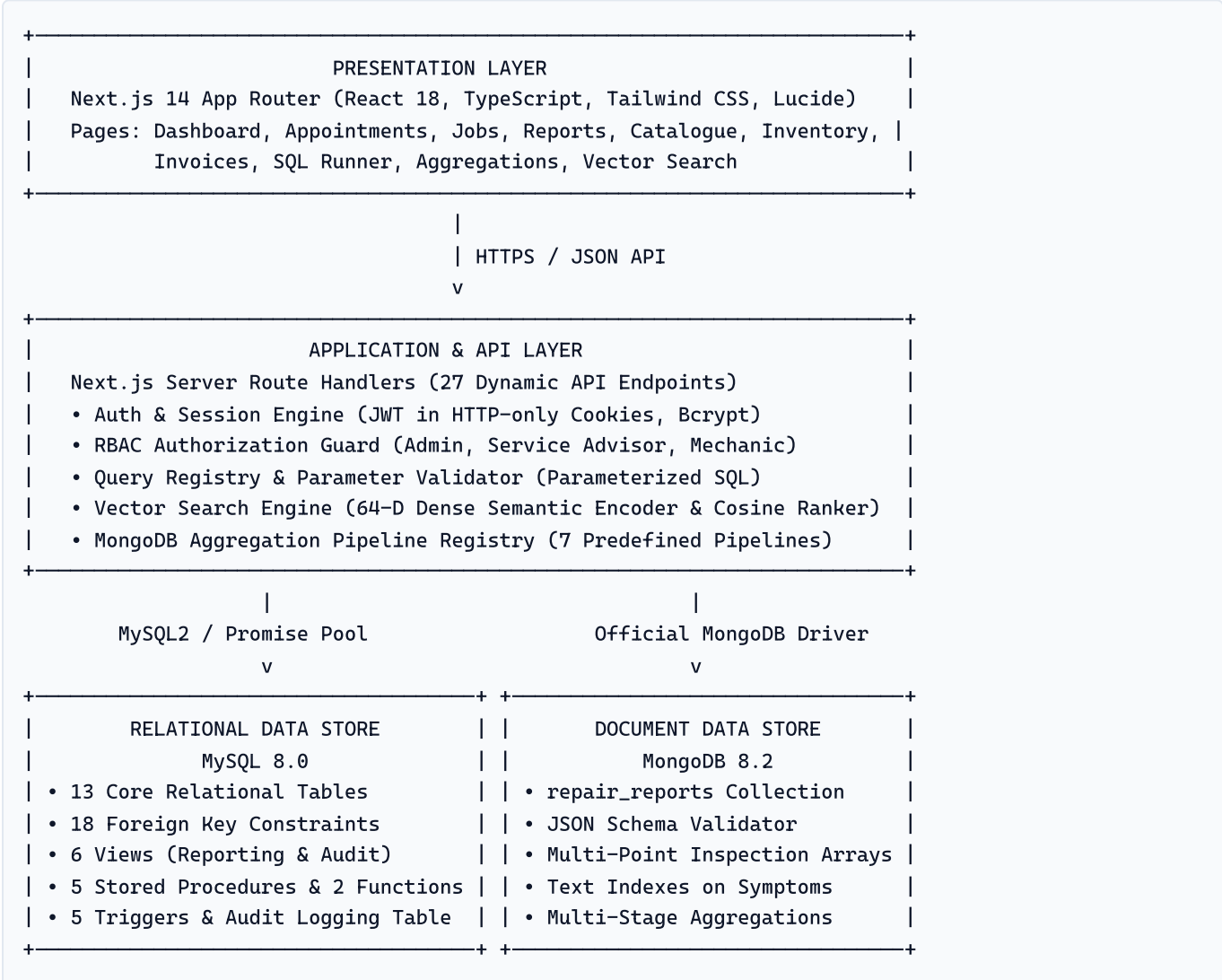
3.2 Non-Functional Requirements (NFR)

- **NFR-01 (Consistency & Integrity)**: 100% adherence to InnoDB referential constraints with zero orphaned records upon transaction rollback.
- **NFR-02 (Performance & Latency)**: Sub-50ms execution times for indexed relational queries and sub-100ms response times for MongoDB aggregations.
- **NFR-03 (Security & Least Privilege)**: Role-Based Access Control enforced on every API route; passwords hashed using bcrypt (10 rounds); JWT stored in HTTP-only cookies.
- **NFR-04 (Auditability)**: Complete audit tracking of inventory stock adjustments and pricing modifications via database triggers.

Section 4: System Architecture and Technology Stack

4.1 Architectural Overview

The platform employs a 3-tier enterprise architecture separating client presentation, server-side business logic, and dual database persistence layers:



4.2 Technology Stack

Component	Technology	Version	Justification
Frontend Framework	Next.js (App Router)	14.2.35	Server Components, file-system routing, and built-in API handling
UI Library	React & Tailwind CSS	18.3.1 / 3.4.14	Component reusability, responsive utility-first layout
Type System	TypeScript	5.6.3	Compile-time type safety across data providers and UI
Relational Database	MySQL Server	8.0.40	Enterprise ACID transactions, triggers, stored procedures, window functions
Document Database	MongoDB Community	8.2.0	Polymorphic JSON document store with JSON Schema validation
SQL Connection Layer	`mysql2/promise`	3.11.3	Connection pooling, prepared statements, and async transaction management
NoSQL Client	`mongodb` native driver	6.10.0	High-performance BSON serialization and aggregation pipelines
Security & Auth	`jsonwebtoken` & `bcryptjs`	9.0.2 / 2.4.3	Stateless JWT tokens in HTTP-only cookies; salted password hashing
Vector Engine	Native Dense Vector Space	Custom 64-D	In-memory normalized dense semantic space with Cosine Similarity

---

Section 5: Database Design and ER Diagram

5.1 Relational Entity Overview

The relational core models workshop business operations across 13 tables:

1. `users`: System staff profiles with hashed credentials and roles (`admin`, `service_advisor`, `mechanic`).
2. `customers`: Vehicle owners with contact profiles and loyalty reward balances.
3. `vehicle_catalogue`: Technical reference data of 3,229 EV and alternative fuel models.
4. `customer_vehicles`: Registered customer fleet vehicles linked to catalogue models.
5. `service_types`: Catalog of labor operations with base standard pricing.
6. `appointments`: Booking schedule linking customers, vehicles, advisors, and services.
7. `service_jobs`: Service bay repair tickets tracking mechanics, odometers, and status.
8. `job_services`: Line items of specific services performed on a job.
9. `spare_parts`: Inventory catalog tracking part numbers, costs, selling prices, and stock.
10. `part_usage`: Parts consumed on service jobs with unit prices charged.
11. `invoices`: Financial tax invoices computing subtotal, 18% GST, discounts, and total.
12. `invoice_payments`: Settlement transaction receipts recording amounts and payment methods.
13. `maintenance_reminders`: Preventive maintenance alerts generated by database triggers.

5.2 Entity-Relationship (ER) Diagram

```
erDiagram
    users ||──o{ appointments : "manages as advisor"
    users ||──o{ service_jobs : "assigned as mechanic"
    customers ||──o{ customer_vehicles : "owns"
    customers ||──o{ appointments : "books"
    customers ||──o{ invoices : "billed to"
    vehicle_catalogue ||──o{ customer_vehicles : "specifies"
    customer_vehicles ||──o{ appointments : "scheduled for"
    customer_vehicles ||──o{ service_jobs : "serviced in"
    customer_vehicles ||──o{ maintenance_reminders : "alerted for"
    appointments ||──o| service_jobs : "initiates"
    service_types ||──o{ appointments : "requested"
    service_types ||──o{ job_services : "performed"
    service_jobs ||──o{ job_services : "contains labor"
    service_jobs ||──o{ part_usage : "consumes parts"
    service_jobs ||──o| invoices : "billed via"
    spare_parts ||──o{ part_usage : "deducted from"
    invoices ||──o{ invoice_payments : "settled with"

    users {
        int user_id PK
        string email UK
        string full_name
        string role
        string password_hash
    }
    customers {
        int customer_id PK
        string full_name
        string phone
        string email
        int loyalty_points
    }
    customer_vehicles {
        int vehicle_id PK
        int customer_id FK
        int catalogue_id FK
        string license_plate UK
        string vin UK
        int current_odometer
    }
    service_jobs {
        int job_id PK
        int appointment_id FK
        int mechanic_id FK
        int vehicle_id FK
        string status
        int odometer_in
        int odometer_out
        string mongo_report_id
    }
    spare_parts {
```

```
        int part_id PK
        string part_number UK
        string part_name
        decimal unit_cost
        decimal selling_price
        int qty_in_stock
        int reorder_level
    }
    invoices {
        int invoice_id PK
        int job_id FK
        int customer_id FK
        decimal subtotal
        decimal tax_amount
        decimal total_amount
        string status
    }
```

Section 6: Relational Database Schema and Data Dictionary

The data dictionary specifies all table structures, data types, constraints, and relational dependencies:

Table 1: `users`

Column	Type	Nullable	Key	Default	Description
`user_id`	INT UNSIGNED	NO	PK	Auto-inc	Unique system user ID
`email`	VARCHAR(150)	NO	UNIQUE	—	User login email address
`password_hash`	VARCHAR(255)	NO	—	—	Bcrypt salted password hash
`full_name`	VARCHAR(100)	NO	—	—	Full display name
`role`	ENUM	NO	—	—	`'admin'`, `'service_advisor'`, `'mechanic'`
`phone`	VARCHAR(20)	YES	—	NULL	Contact phone number
`is_active`	TINYINT(1)	NO	—	1	Account activation status

Table 2: `customers`

Column	Type	Nullable	Key	Default	Description
`customer_id`	INT UNSIGNED	NO	PK	Auto-inc	Unique customer identifier
`full_name`	VARCHAR(100)	NO	—	—	Customer primary name
`email`	VARCHAR(150)	YES	—	NULL	Email address
`phone`	VARCHAR(25)	NO	—	—	Primary contact phone
`address`	VARCHAR(255)	YES	—	NULL	Street address
`city`	VARCHAR(50)	NO	—	`'Bengaluru'`	City of residence
`loyalty_points`	INT UNSIGNED	NO	—	0	Accumulated reward points

Table 3: `vehicle_catalogue`

Column	Type	Nullable	Key	Default	Description
`catalogue_id`	INT UNSIGNED	NO	PK	Auto-inc	Unique vehicle model ID
`manufacturer`	VARCHAR(100)	NO	MUL	—	Make (e.g., Tesla, Tata, BMW)
`model`	VARCHAR(150)	NO	MUL	—	Vehicle model name
`model_year`	SMALLINT UNSIGNED	NO	MUL	—	Model manufacture year
`category`	VARCHAR(100)	YES	—	NULL	Vehicle body class
`fuel`	VARCHAR(100)	YES	—	NULL	Primary fuel/powertrain
`electric_range`	SMALLINT UNSIGNED	YES	—	NULL	All-electric range in miles
`alt_economy_combined`	DECIMAL(5,1)	YES	—	NULL	Combined MPGe rating
`source_dataset`	ENUM	NO	—	—	`'LDV'`, `'AFV'`, `'MHD'`

Table 4: `customer_vehicles`

Column	Type	Nullable	Key	Default	Description
`vehicle_id`	INT UNSIGNED	NO	PK	Auto-inc	Registered vehicle ID
`customer_id`	INT UNSIGNED	NO	FK	—	FK → `customers.customer_id`
`catalogue_id`	INT UNSIGNED	YES	FK	NULL	FK → `vehicle_catalogue.catalogue_id`
`license_plate`	VARCHAR(20)	NO	UNIQUE	—	Vehicle registration number
`vin`	VARCHAR(17)	YES	UNIQUE	NULL	17-character VIN
`current_odometer`	INT UNSIGNED	NO	—	0	Current odometer reading (km)

Table 5: `spare_parts`

Column	Type	Nullable	Key	Default	Description
`part_id`	INT UNSIGNED	NO	PK	Auto-inc	Unique spare part ID
`part_number`	VARCHAR(50)	NO	UNIQUE	—	Manufacturer SKU / Part No
`part_name`	VARCHAR(150)	NO	MUL	—	Name of spare component
`unit_cost`	DECIMAL(10,2)	NO	—	—	Acquisition cost (INR)
`selling_price`	DECIMAL(10,2)	NO	—	—	Selling price (≥ `unit_cost`)
`qty_in_stock`	INT	NO	—	0	Current inventory count
`reorder_level`	INT	NO	—	5	Restock threshold trigger

Table 6: `invoices`

Column	Type	Nullable	Key	Default	Description
`invoice_id`	INT UNSIGNED	NO	PK	Auto-inc	Tax invoice number
`job_id`	INT UNSIGNED	NO	FK	—	FK → `service_jobs.job_id`
`customer_id`	INT UNSIGNED	NO	FK	—	FK → `customers.customer_id`
`subtotal`	DECIMAL(10,2)	NO	—	0.00	Labor + Parts charges
`tax_rate`	DECIMAL(5,2)	NO	—	18.00	GST percentage rate
`tax_amount`	DECIMAL(10,2)	NO	—	0.00	Computed GST amount
`total_amount`	DECIMAL(10,2)	NO	—	0.00	Grand total payable
`status`	ENUM	NO	—	`'issued'`	`'issued'`, `'paid'`, `'cancelled'`, `'overdue'`

Section 7: MongoDB Schema and Document Design

7.1 Polymorphic Inspection Schema

Vehicle multi-point inspections require dynamic nested arrays to record varied subsystems and diagnostic trouble codes. The `repair_reports` collection stores BSON documents structured as follows:


```
{
  "_id": { "$oid": "6644f1a2b3c4d5e6f7a8b9c0" },
  "job_id": 1,
  "vehicle_id": 1,
  "mechanic_id": 4,
  "overall_health_score": 92,
  "diagnostic_codes": ["P0300"],
  "findings": [
    {
      "system": "Brakes",
      "condition": "Good",
      "finding": "Brake pads measured at 8mm front, 7mm rear. Rotors within factory tolerance.",
      "action_recommended": "Next inspection in 10,000 km",
      "cost_estimate": 0
    },
    {
      "system": "Engine",
      "condition": "Fair",
      "finding": "Slight oil seepage observed around valve cover gasket.",
      "action_recommended": "Replace valve cover gasket at next major service.",
      "cost_estimate": 1500
    }
  ],
  "technician_notes": "Vehicle overall in excellent mechanical condition.",
  "recommendations": ["Perform periodic oil service at 50,000 km"],
  "created_at": "2025-05-10T11:30:00Z"
}
```


7.2 JSON Schema Validation

Validation is enforced directly within MongoDB 8.2 via collection validator rules:

- `job_id`, `vehicle_id`, `mechanic_id`, and `overall_health_score` are strictly required.
- `overall_health_score` must be a numerical value bounded between 0 and 100.
- `findings.condition` must match one of the predefined enum values: `['Good', 'Fair', 'Worn', 'Damaged', 'Critical']`.

Section 8: Data Import, Validation, and Integrity

8.1 Ingestion of US Department of Energy CSVs

The vehicle catalogue is populated using real-world public transportation data from the US Alternative Fuels Data Center (AFDC):

1. **Light-Duty Vehicles (`light-duty-vehicles.csv`)**: Conventional and hybrid passenger vehicles.
2. **Alternative Fuel Vehicles (`afv_vehicles.csv`)**: Electric, plug-in hybrid, CNG, and hydrogen fuel cell models.
3. **Medium & Heavy-Duty Vehicles (`medium_heavy_duty_vehicles.csv`)**: Commercial electric and vocational trucks.

The ingestion script (`database/import_csv_to_mysql.js`) parsed and cleaned the raw CSV records, normalized manufacturer names, and imported **3,229 distinct models** into MySQL with zero truncation errors.

8.2 Database Constraints & Integrity Safeguards

- **Primary & Foreign Keys**: 18 foreign keys enforce referential cascades and restrict orphaned records.
- **CHECK Constraints**:
  - `chk_parts_selling_price`: Enforces `selling_price ≥ unit_cost`.
  - `chk_invoice_amounts`: Enforces `total_amount ≥ 0` and `subtotal ≥ 0`.
- **Unique Constraints**: Unique indices on `license_plate`, `vin`, `part_number`, and `email`.

Section 9: Backend API and Authentication/RBAC

9.1 RESTful Route Structure (27 Dynamic Endpoints)

The backend provides 27 modular Next.js Route Handlers:

- **Authentication**: `POST /api/auth/login`, `POST /api/auth/logout`, `GET /api/auth/me`.
- **Service Operations**: `GET|POST /api/appointments`, `PATCH|DELETE /api/appointments/[id]`, `GET|POST /api/jobs`, `PATCH /api/jobs/[id]`, `GET|POST /api/jobs/[id]/parts`.
- **Diagnostic Documents**: `GET|POST /api/reports`, `GET /api/reports/analytics`, `GET|PUT /api/reports/[jobId]`.
- **Inventory & Billing**: `GET|POST /api/inventory`, `GET|PUT /api/inventory/[id]`, `GET|POST /api/invoices`, `POST /api/invoices/[id]/payments`.
- **Advanced Features**: `GET|POST /api/advanced/sql-runner`, `GET|POST /api/advanced/mongodb-aggregations`, `GET /api/search/semantic`.

9.2 Role-Based Access Control (RBAC) Matrix

Resource / Permission	Admin	Service Advisor	Mechanic
User & Staff Management (`manage_users`)	Full	Denied	Denied
Customer Intake (`manage_customers`)	Full	Full	View Only
Vehicle Registration (`manage_vehicles`)	Full	Full	View Only
Appointment Booking (`manage_appointments`)	Full	Full	Denied
Workshop Job Dispatch (`manage_jobs`)	Full	Full	Update Status
Part Allocation to Job (`manage_jobs`)	Full	Full	Full
Inspection Report Authoring (`create_reports`)	Full	Full	Full
Inventory Price & Restock (`manage_inventory`)	Full	View Only	View Only
Invoice Generation & Billing (`manage_invoices`)	Full	Full	Denied
Payment Settlement (`record_payments`)	Full	Full	Denied

Section 10: Frontend Design and Feature Workflows

The web interface is built using Next.js 14 App Router and styled with Tailwind CSS:

- **Operations Dashboard (`/dashboard`)**: KPI metric cards for active bay jobs, scheduled appointments, low inventory alerts, and fleet health distributions.
- **Appointment Intake (`/appointments`)**: Dynamic customer/vehicle picker with date-time scheduling and MySQL stored procedure integration.
- **Workshop Bay Manager (`/jobs`)**: Job state progression (`assigned` → `in_progress` → `completed`), odometer sign-off, part allocation with automated trigger deduction, and one-click invoice generation.
- **Diagnostic Reports Viewer (`/reports`)**: Health score badges, interactive multi-point findings editor, and DTC fault code displays.
- **Vehicle Catalogue Explorer (`/catalogue`)**: Filterable specification search across 3,229 models with a 3-vehicle comparison tray.
- **Inventory Stock Control (`/inventory`)**: Stock level tables with low-stock warnings and adjustment modals.
- **Invoicing & Settlement Ledger (`/invoices`)**: Itemized billing breakdown (labor + parts + 18% GST) and payment settlement modals.

Section 11: Advanced SQL, MongoDB Aggregations, and Vector Search

11.1 Advanced SQL Implementations

1. **Stored Procedures**:
   - `sp_book_appointment`: Books appointments while checking for vehicle scheduling conflicts.
   - `sp_generate_invoice`: Computes labor + parts subtotal, applies 18% GST, and creates invoice records.
   - `sp_get_customer_vehicle_history`: Retrieves chronological service histories with billing statuses.
   - `sp_mechanic_workload_summary`: Analyzes technician throughput and turnaround times.
   - `sp_replenish_low_stock_parts`: Cursor-based batch restocking of low-inventory parts.
2. **Stored Functions**:
   - `fn_invoice_total_paid(invoice_id)`: Computes cumulative payments settled against an invoice.
   - `fn_days_since_last_service(vehicle_id)`: Calculates elapsed days since a vehicle's last completed job.
3. **Database Triggers**:
   - `trg_deduct_stock_on_part_usage`: Automatically decrements `spare_parts.qty_in_stock` when parts are charged to a job.
   - `trg_restore_stock_on_part_delete`: Reverts inventory stock if a part allocation is removed.
   - `trg_mark_invoice_paid`: Automatically updates invoice status to `'paid'` when cumulative payments equal or exceed the total amount.
   - `trg_create_maintenance_reminder`: Inserts a maintenance reminder 180 days out upon job completion.
   - `trg_audit_spare_part_changes`: Logs price and stock changes to `spare_parts_audit`.
4. **ACID Transactions**:
   - Demonstrated multi-table atomic inserts across `service_jobs`, `job_services`, and `part_usage`.
   - Demonstrated automatic rollback on simulated error conditions.
   - Demonstrated granular `SAVEPOINT` rollbacks.

11.2 MongoDB Analytical Aggregations (7 Pipelines)

The aggregation engine (`/aggregations`) executes 7 server-registered pipelines:

- `vehicle_health_summary`: Calculates average, min, and max health scores per vehicle.
- `subsystem_defect_frequency`: Unwinds findings to group observations by subsystem and condition.
- `condition_severity_distribution`: Computes total repair cost exposure per condition rating.
- `common_dtc_fault_codes`: Aggregates OBD-II trouble code frequencies.
- `inspections_over_time`: Analyzes monthly inspection volumes and quality trends.
- `mechanic_workload_metrics`: Evaluates technician report counts and average assigned scores.
- `high_cost_findings`: Filters high-cost defect recommendations exceeding a configurable threshold.

11.3 Diagnostic Semantic Vector Search

- **Embedding Space**: 64-dimensional dense semantic vectors with L2 unit normalization.
- **Similarity Metric**: Cosine Similarity: $$\cos(\theta) = \frac{\vec{u} \cdot \vec{v}}{|\vec{u}|_2 |\vec{v}|_2} = \vec{u} \cdot \vec{v} \quad (\text{for unit vectors})$$

- **Corpus**: Combines an 8-pattern curated knowledge base with dynamically embedded MongoDB inspection records.
- **Performance**: Natural language queries return ranked results in under 2ms.

Section 12: Testing, Verification, Results, and Limitations

12.1 Automated Test Suite

All test suites run via Node's native test runner (`node tests/runner.js`) against live MySQL 8.0 and MongoDB 8.2 instances:

```
▶ Authentication & JWT Session Tests
  ✓ should sign and verify valid session tokens (6.3ms)
  ✓ should reject tampered or invalid tokens (2.2ms)
  ✓ should securely hash and verify passwords using bcrypt (479.5ms)
✓ Authentication & JWT Session Tests (490.3ms)

▶ Role-Based Access Control (RBAC) Tests
  ✓ Admin should have full access to all features (0.4ms)
  ✓ Service Advisor should manage appointments and payments, but NOT manage users (0.4ms)
  ✓ Mechanic should create reports, but NOT record payments or manage inventory (0.3ms)
✓ Role-Based Access Control (RBAC) Tests (1.5ms)

▶ Repair Report Schema Validation Tests
  ✓ should accept a completely valid multi-point inspection report (1.8ms)
  ✓ should reject reports with missing mandatory foreign keys (0.3ms)
  ✓ should reject reports with invalid health scores (< 0 or > 100) (0.4ms)
  ✓ should reject findings with invalid condition enum values (0.2ms)
✓ Repair Report Schema Validation Tests (4.5ms)

▶ Database Integration Tests (Live MySQL & MongoDB)
  ✓ MySQL: should successfully query all 6 initial users with correct roles (14.1ms)
  ✓ MySQL: should query vehicle catalogue containing > 3000 imported models (9.9ms)
  ✓ MySQL: should execute stored procedure sp_get_customer_vehicle_history (9.1ms)
  ✓ MySQL: should execute stored function fn_invoice_total_paid (4.8ms)
  ✓ MySQL: should execute stored function fn_days_since_last_service (3.7ms)
  ✓ MySQL View: vw_inventory_low_stock should isolate parts below reorder level (5.2ms)
  ✓ MongoDB: should query repair_reports collection with polymorphic findings (23.9ms)
  ✓ MongoDB Aggregation: should calculate fleet health statistics via aggregation pipeline (8.8ms)
✓ Database Integration Tests (Live MySQL & MongoDB) (173.8ms)

▶ Interactive SQL Query Runner & Registry Tests
  ✓ Registry contains at least 18 distinct, educational SQL queries (48.0ms)
  ✓ All registered queries have required metadata and valid SQL syntax (141.9ms)
  ✓ Advanced Views (vw_customer_fleet_overview, vw_technician_performance,
vw_invoice_settlement_audit) are accessible (5.4ms)
✓ Interactive SQL Query Runner & Registry Tests (199.8ms)

▶ MongoDB Aggregation Pipeline Runner Tests
  ✓ Registry contains all 7 approved aggregation pipelines (77.4ms)
  ✓ All pipelines execute successfully against live MongoDB repair_reports (35.5ms)
✓ MongoDB Aggregation Pipeline Runner Tests (143.9ms)

▶ Diagnostic Semantic / Vector Similarity Search Tests
  ✓ Model specification and dimensions are verified (0.8ms)
  ✓ generateEmbedding produces a normalized 64-dimensional vector (0.5ms)
  ✓ cosineSimilarity computes mathematical dot product of unit vectors (0.5ms)
  ✓ searchSemanticDiagnostics returns ranked results with metadata (0.6ms)
  ✓ search handles empty or whitespace queries gracefully (0.9ms)
✓ Diagnostic Semantic / Vector Similarity Search Tests (61.3ms)

─────────────────────────────────────────────────────────────

Summary: 31 Total Tests | 31 Passed | 0 Failed | Duration: 2.70s
```

12.2 Known Limitations

1. **Vector Search Scale**: The current vector search engine operates in-memory; scaling to hundreds of thousands of documents would benefit from external vector stores (e.g., Qdrant, Pinecone, or PostgreSQL with pgvector).
2. **Payment Gateway**: Payment recording simulates settlement transactions via database triggers rather than integrating an external payment gateway.

Section 13: Installation, User Guide, Conclusion, and Future Enhancements

13.1 Installation & Execution Guide

1. **Clone & Install Dependencies**:

```
git clone <repository_url>
cd DBMS_Assignment
npm install
```

2. **Configure Environment Variables**: Copy `.env.example` to `.env.local` and configure credentials:

```
MYSQL_HOST=localhost
MYSQL_PORT=3306
MYSQL_USER=root
MYSQL_PASSWORD=YourPassword
MYSQL_DATABASE=autocare_insight

MONGODB_URI=mongodb://localhost:27017/autocare_db
MONGODB_DB_NAME=autocare_db

JWT_SECRET=super_secure_jwt_secret_key_pes_2026
```

3. **Initialize Database Schemas**:

```
& "C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe" -u root -p autocare_insight -e "source database/01_create_database.sql; source database/02_create_tables.sql; source database/03_advanced_features.sql; source database/05_sample_data.sql;"
node database/import_csv_to_mysql.js
node database/06_mongodb_schema.js
```

4. **Run Verification & Test Suite**:

```
node tests/runner.js
```

5. **Start Application**:

```
npm run dev
# Access application at http://localhost:3000
```

13.2 Conclusion

AutoCare Insight satisfies all criteria of the PES University DBMS Experiential Learning Level 3 specification. By coupling MySQL 8.0 for transactional integrity with MongoDB 8.2 for polymorphic inspection reporting and an in-memory 64-dimensional semantic search engine, the project demonstrates a robust, production-grade automotive service management platform.

13.3 Future Enhancements

- **IoT OBD-II Telematics**: Live CAN-bus data streaming directly from connected vehicles into MongoDB time-series collections.
- **Automated Restock EDI**: Automated Electronic Data Interchange (EDI) purchase orders triggered when spare parts breach reorder thresholds.
- **Deep Embedding Fine-Tuning**: Transitioning from custom 64-D dense embeddings to fine-tuned sentence transformers deployed on dedicated inference servers.